Лабораторная работа № 23: Исследование методов работы с MongoDB средствами Node.js.

Цель работы: Сформировать умение анализировать методы работы с MongoDB средствами Node.js.

Задание 1 (Базовый уровень)

Подключение к MongoDB и выполнение простого запроса

Описание:
Необходимо установить соединение с MongoDB через официальный драйвер и выполнить простой запрос. Продемонстрировать разницу между одиночным подключением и `MongoClient`.

Как это должно работать:

1. Настройка подключения:
    - URI подключения из переменных окружения (`.env`)
    - Формат: `mongodb://student:password@localhost:27017/bbmo_01_23`
    - Имя базы: `bbmo_01_23` (создаётся лениво при первой вставке)
2. Подключение через `MongoClient`:
    - `client.connect()` — установка соединения с кластером
    - `client.db(dbName)` — доступ к БД (легковесная операция)
    - `db.command({ ping: 1 })` — проверка соединения
    - `client.close()` — закрытие соединения
3. Информация о сервере:
    - Версия MongoDB (`buildInfo`)
    - Список баз данных (`admin.listDatabases()`)
    - Список коллекций в текущей БД
4. Heartbeat и переподключение:
    - Демонстрация периодических проверок доступности
    - Логирование событий `serverHeartbeatSucceeded`, `serverHeartbeatFailed`
    - Автоматическое переподключение при потере связи

Пример работы:

```
text
```

```
$ node mongodb-connect.js

=== Подключение к MongoDB ===
✔ Подключение установлено
Версия MongoDB: 7.0.5
База данных: bbmo_01_23
Пользователь: student

=== Информация о сервере ===
Список БД:
  - admin
  - bbmo_01_23
  - local

Коллекции в bbmo_01_23:
  (пусто — БД создана лениво)

=== Heartbeat ===
[HEARTBEAT] Успешно: 2024-01-15T14:30:25.123Z
[HEARTBEAT] Успешно: 2024-01-15T14:30:35.456Z
✔ Соединение стабильно
```

```
✖ Соединение закрыто
```

Что нужно сделать для отчёта:

- Сделать скриншот успешного подключения
- Сделать скриншот информации о сервере
- Сделать скриншот событий heartbeat
- Сделать скриншот ошибки подключения (неверный URI)

Задание 2 (Средний уровень)

CRUD-операции в MongoDB

Описание:
Выполнить базовые CRUD-операции с коллекцией `students`: вставка, поиск, обновление, удаление.

Как это должно работать:

1. Коллекция `students`:
    - Документы вида:
    - `json`

```
{
  "_id": ObjectId,
  "name": "Иван Иванов",
  "group_name": "ББМО-01-23",
  "course": 2,
  "grade": 4.5,
  "created_at": ISODate
```

    - `}`
    - Индекс по `group_name`
2. INSERT:
    - `insertOne(doc)` — вставка одного документа
    - `insertMany([...])` — вставка нескольких
    - Проверка `insertedId` и `insertedIds`
    - Группа студентов — ББМО-01-23
3. SELECT (find):
    - `find({})` — все документы (возвращает курсор)
    - `find({ group_name: "ББМО-01-23" })` — с фильтром
    - `find().sort({ grade: -1 })` — с сортировкой
    - `find().limit(10).skip(20)` — с пагинацией
    - `findOne({ _id: ... })` — один документ
    - Важно: `toArray()` загружает всё в память — опасно для больших данных!
4. UPDATE:
    - `updateOne(filter, update)` — обновление одного
    - `updateMany(filter, update)` — обновление многих
    - Операторы: `$set`, `$inc`, `$unset`, `$push`
    - Проверка `matchedCount` и `modifiedCount`
5. DELETE:
    - `deleteOne(filter)` — удаление одного
    - `deleteMany(filter)` — удаление многих
    - Проверка `deletedCount`
6. Проекции:
    - `find({}, { projection: { name: 1, grade: 1, _id: 0 } })`
    - Уменьшение сетевого трафика
    - Сравнение с полной выборкой

Пример работы:

```
text
```

```
$ node mongodb-crud.js

=== Коллекция students (группа ББМО-01-23) ===

--- INSERT ---
✔ Вставлен: insertedId=ObjectId("...")
✔ Вставлено 3 документа: insertedIds={...}

--- SELECT ---
Все документы (10):
  { _id: ..., name: "Иван", group: "ББМО-01-23", grade: 4.5 }
  ...

С фильтром по группе (5):
  { name: "Иван", group: "ББМО-01-23", grade: 4.5 }
  ...

С проекцией (только name, grade):
  { name: "Иван", grade: 4.5 }
  { name: "Мария", grade: 3.8 }
  ...

--- UPDATE ---
✔ updateOne: matchedCount=1, modifiedCount=1
✔ updateMany (group=ББМО-01-23, $inc: course+1): matchedCount=5

--- DELETE ---
✔ deleteOne: deletedCount=1
```

```
✔ deleteMany (grade < 3): deletedCount=3
```

Что нужно сделать для отчёта:

- Скриншот INSERT с `insertedId`
- Скриншот `find` с фильтром
- Скриншот `find` с сортировкой и пагинацией
- Скриншот `updateOne` и `updateMany`
- Скриншот `deleteOne` и `deleteMany`
- Скриншот проекции

Задание 3 (Продвинутый уровень)

Курсоры, агрегации и работа с большими данными

Описание:
Изучить работу с курсорами, агрегационный pipeline и потоковую обработку больших коллекций.

Как это должно работать:

1. Курсоры:
    - `find()` возвращает курсор, а не данные
    - Итерация через `for await (const doc of cursor)`
    - `cursor.toArray()` — загрузка всего (опасно)
    - `cursor.next()` — по одному
    - `cursor.batchSize(n)` — размер порции
    - Закрытие курсора: `cursor.close()`
2. Агрегации (Aggregation Pipeline):
    - `db.collection.aggregate([...])`
    - Этапы: `$match`, `$group`, `$sort`, `$project`, `$limit`, `$lookup`
    - Примеры:
        - Средний балл по группам
        - Количество студентов по курсам
        - Топ-10 студентов по оценке
        - JOIN с коллекцией `groups` через `$lookup`
3. Потоковая обработка:
    - Генерация 1 000 000 документов
    - Чтение через курсор с `batchSize: 1000`
    - Обработка каждого документа
    - Сравнение памяти: `toArray()` vs итерация
4. Индексы:
    - Создание индекса: `createIndex({ group_name: 1 })`
    - Составной индекс: `createIndex({ group_name: 1, grade: -1 })`
    - Уникальный индекс: `createIndex({ email: 1 }, { unique: true })`
    - Проверка использования: `explain("executionStats")`
    - Сравнение времени запроса с индексом и без
5. Текстовый поиск:
    - Создание текстового индекса: `createIndex({ name: "text" })`
    - Поиск: `find({ $text: { $search: "Иван" } })`
    - Ранжирование по релевантности

Пример работы:

```
text
```

```
$ node mongodb-advanced.js

=== Курсоры ===
Итерация через for await:
  Получено 10000 документов за 1.2 сек
Пик памяти: 35 МБ

Через toArray():
✖ Ошибка: JavaScript heap out of memory
  Требуется ~800 МБ памяти

=== Агрегации ===
Средний балл по группам:
  ББМО-01-23: 4.12
  ББМО-02-23: 3.95
  ББМО-03-23: 4.08

Количество по курсам:
  1 курс: 300
  2 курс: 350
  3 курс: 200
  4 курс: 150

$lookup (JOIN с groups):
  { group_name: "ББМО-01-23", students: 40, curator: "Иванов И.И." }
  ...

=== Индексы ===
✔ Создан индекс: group_name_1
✔ Создан индекс: group_name_1_grade_-1
✔ Создан уникальный индекс: email_1

explain() без индекса:
  executionTimeMillis: 245
  totalDocsExamined: 100000

explain() с индексом:
  executionTimeMillis: 3
  totalDocsExamined: 40
  ✔ Ускорение в 80 раз

=== Текстовый поиск ===
Создан текстовый индекс: name_text
find({ $text: { $search: "Иван" } }):
  Найдено: 15 документов
```

```
Топ-1: score=1.5, name="Иван Иванов"
```

Что нужно сделать для отчёта:

- Скриншот итерации через `for await`
- Скриншот ошибки при `toArray()`
- Скриншот агрегаций (средний балл, количество)
- Скриншот `$lookup`
- Скриншот создания индексов
- Скриншот `explain()` без индекса и с индексом
- Скриншот текстового поиска

Задание 4 (Сложный уровень)

Транзакции, реплика-сет и Change Streams

Описание:
Изучить транзакции в MongoDB (v4+), работу с реплика-сетом и Change Streams для отслеживания изменений в реальном времени.

Как это должно работать:

1. Настройка реплика-сета:
    - Запуск MongoDB с `--replSet rs0`
    - Инициализация: `rs.initiate()`
    - Проверка статуса: `rs.status()`
    - Транзакции работают только в реплика-сете!
2. Транзакции:
    - `client.startSession()` — начало сессии
    - `session.startTransaction()` — начало транзакции
    - `collection.insertOne(doc, { session })` — с сессией
    - `session.commitTransaction()` или `session.abortTransaction()`
    - `session.endSession()` в `finally`
    - Важно: многоверсионные транзакции (MVCC)
3. Демонстрация:
    - Успешная транзакция (2 INSERT + 1 UPDATE)
    - Откат при ошибке (дубликат)
    - Проверка, что данные не сохранились после отката
4. Change Streams:
    - `collection.watch()` — отслеживание изменений

    - События: `insert`, `update`, `delete`, `replace`
    - Обработка через `for await (const change of stream)`
    - Использование в реальном времени (например, WebSocket)
5. Демонстрация:
    - Открытие Change Stream на коллекции `students`
    - Параллельно выполняются INSERT, UPDATE, DELETE
    - Вывод всех событий в консоль
    - Закрытие stream при завершении
6. Read Preference:
    - `primary` — чтение с primary (по умолчанию)
    - `primaryPreferred`, `secondary`, `secondaryPreferred`, `nearest`
    - Влияние на консистентность и нагрузку

Пример работы:

```
text

$ node mongodb-transactions.js

=== Реплика-сет ===
✔ Реплика-сет: rs0
✔ Primary: localhost:27017
✔ Вторичные: нет (один узел)

=== Транзакция (успех) ===
[SESSION] Начало
[BEGIN] Транзакция начата
[INSERT] Студент 1: Иван
[INSERT] Студент 2: Мария
[UPDATE] Студент 1: grade=4.8
[COMMIT] Транзакция зафиксирована
✔ Данные сохранены

=== Транзакция (откат) ===
[BEGIN] Транзакция начата
[INSERT] Студент 3: Петр
[INSERT] Студент 3 (дубликат): ✖ ER_DUP_ENTRY
[ABORT] Транзакция отменена
✔ Данные не сохранены
Проверка: студентов в БД = 2 (Петр не добавлен)

=== Change Streams ===
[WATCH] Отслеживание коллекции students
[CHANGE] insert: { name: "Анна" }
[CHANGE] update: { _id: ..., grade: 5.0 }
```

```
[CHANGE] delete: { _id: ... }
^C
[WATCH] Stream закрыт

=== Read Preference ===
primary:             5 мс
secondaryPreferred:  3 мс (если есть вторичные)
nearest:             2 мс
```

Что нужно сделать для отчёта:

- Скриншот настройки реплика-сета
- Скриншот успешной транзакции
- Скриншот отката транзакции
- Скриншот проверки данных после отката
- Скриншот Change Stream с событиями
- Скриншот Read Preference

Задание 5 (Экспертный уровень)

Полноценный слой доступа к данным (DAO) с MongoDB + Koa.js

Описание:
Создать полнофункциональный DAO для MongoDB и REST API на Koa.js с транзакциями, агрегациями, Change Streams и мониторингом.

Как это должно работать:

1. Структура проекта:
    - `config/` — конфигурация MongoDB
    - `models/` — модели (схемы, валидация)
    - `dao/` — Data Access Objects
    - `services/` — бизнес-логика
    - `controllers/` — контроллеры
    - `middleware/` — БД, ошибки, логирование
    - `routes/` — маршруты
    - `streams/` — Change Streams обработчики
2. Класс `StudentDAO`:
    - `create(data)` — с валидацией
    - `findById(id)` — с `ObjectId`

    - `findAll(options)` — пагинация, сортировка, фильтры
    - `update(id, data)` — с `$set`
    - `delete(id)`
    - `search(query)` — текстовый поиск
    - `getStats()` — агрегация
    - `batchCreate(array)` — массовая вставка
3. REST API:
    - `GET /api/students` — список с пагинацией
    - `GET /api/students/:id` — один
    - `POST /api/students` — создание
    - `PUT /api/students/:id` — обновление
    - `DELETE /api/students/:id` — удаление
    - `GET /api/students/stats` — агрегация
    - `GET /api/students/search?q=...` — поиск
    - `POST /api/students/batch` — массовая вставка с транзакцией
4. Транзакции:
    - Массовая вставка через сессию
    - Откат при ошибке
    - `session.endSession()` в `finally`
5. Change Streams + WebSocket:
    - Отслеживание изменений коллекции `students`
    - Отправка уведомлений через WebSocket
    - Клиент получает события в реальном времени
6. Мониторинг:
    - Логирование всех операций
    - Метрики: количество запросов, среднее время, ошибки
    - `GET /api/metrics`
7. Кэширование:
    - In-memory кэш для GET-запросов
    - TTL: 60 секунд
    - Инвалидация при изменении
8. Валидация:
    - JSON Schema для документов
    - Проверка на уровне приложения
    - Проверка на уровне MongoDB (валидатор коллекции)

Пример работы:

```
text
```

```
$ npm start
[INFO] MongoDB подключена: bbmo_01_23
[INFO] Индексы созданы: group_name_1, name_text
```

```
[INFO] Change Stream запущен
[INFO] Koa-сервер на порту 3000
[INFO] WebSocket на порту 3001

$ curl http://localhost:3000/api/students?limit=3
{
  "data": [
    {"_id": "...", "name": "Иван", "group_name": "ББМО-01-23", "grade":
4.5},
    {"_id": "...", "name": "Мария", "group_name": "ББМО-01-23", "grade":
3.8},
    {"_id": "...", "name": "Петр", "group_name": "ББМО-01-23", "grade":
4.2}
  ],
  "pagination": {"total": 1000, "limit": 3, "offset": 0}
}

$ curl -X POST http://localhost:3000/api/students/batch \
  -H "Content-Type: application/json" \
  -d '[{"name": "Анна", ...}, {"name": "Олег", ...}]'
{
  "inserted": 2,
  "transaction": "committed"
}

$ curl http://localhost:3000/api/students/stats
{
  "total": 1002,
  "averageGrade": 4.11,
  "byGroup": {"ББМО-01-23": 42, "ББМО-02-23": 35, "ББМО-03-23": 25}
}

$ curl "http://localhost:3000/api/students/search?q=Иван"
{
  "found": 15,
  "data": [{"name": "Иван Иванов", "score": 1.5}, ...]
}

# WebSocket клиент:
[WS] Подключен
[WS] insert: { name: "Анна" }
[WS] update: { _id: ..., grade: 5.0 }
[WS] delete: { _id: ... }

# Логи
[2024-01-15 14:30:25] [ББМО-01-23] find students (1.2ms)
```

```
[2024-01-15 14:30:28] [ББМО-01-23] BEGIN TRANSACTION
[2024-01-15 14:30:28] [ББМО-01-23] insertMany (2.3ms)
[2024-01-15 14:30:28] [ББМО-01-23] COMMIT
```

Что нужно сделать для отчёта:

- Скриншот структуры проекта
- Скриншот запуска сервера
- Скриншот `GET /api/students` с пагинацией
- Скриншот `POST /api/students/batch` с транзакцией
- Скриншот `GET /api/students/stats` (агрегация)
- Скриншот поиска с текстовым индексом
- Скриншот WebSocket-уведомлений
- Скриншот Change Stream в консоли
- Скриншот `GET /api/metrics`
- Скриншот логов

Фиксация изменений

Добавить файлы в Git
Создать коммит
Отправить в удалённый репозиторий
Создать Pull Request